

# Procedures, Stack Frames, and Calling Conventions

Function calls, return addresses, register preservation, stack frames, and AArch64 procedure structure

Computer Organization & Architecture | AArch64 Reference

# Topic 8 Outline

## Topic Objective

Explain how AArch64 implements function calls using branch-with-link, return addresses, stack discipline, and register-preservation rules.

- **Section 1:** Procedure calls and returns
- **Section 2:** Argument passing and return values
- **Section 3:** Caller-saved and callee-saved registers
- **Section 4:** Stack frames, prologues, and epilogues
- **Section 5:** Leaf functions, non-leaf functions, and recursion
- **Section 6:** Summary and self-test

# Procedure Calls and Returns

# Why Procedures Need a Convention

## Problem

A function call temporarily transfers control to another code region, but the caller must later continue as if the call produced one well-defined result.

- The caller and callee need an agreed way to pass arguments.
- The return address must be saved somewhere.
- Both sides must know which registers may be overwritten.
- The stack must remain correctly aligned and balanced.

## Calling Convention

The calling convention is the software contract that makes separately compiled functions able to call each other correctly.

# Branch with Link

## Function Call Instruction

`bl function`

- `bl` branches to the target label.
- The address of the next instruction is written to `x30`.
- Register `x30` is also called the link register, `lr`.
- The callee uses that saved address to return to the caller.

## Architectural Effect

`bl` changes control flow and records the return address. It does not automatically save general-purpose registers or allocate a stack frame.

# Return from a Procedure

## Return Instruction

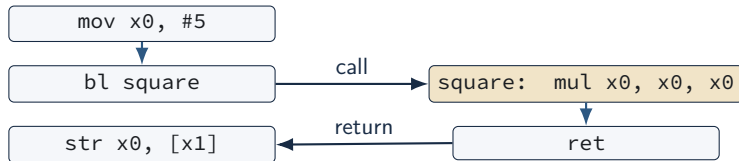
`ret`

- By default, `ret` branches to the address held in `x30`.
- It completes the control-flow path started by `bl`.
- The return value, if any, is usually already placed in `x0`.
- The stack pointer must be restored before returning.

## Common Form

Most simple returns use `ret`. The explicit form `ret x30` means return to the address in `x30`.

# Call and Return Control Flow



## Control-Flow Rule

The call target runs temporarily, then `ret` resumes execution after the original `bl` instruction.

# The Link Register Hazard

## Why Non-Leaf Functions Need Care

A function that calls another function will execute its own `bl`. That new `bl` overwrites `x30`.

### Leaf Function

- Calls no other function.
- May return using the original `x30`.
- May not need a stack frame.

### Non-Leaf Function

- Calls at least one function.
- Must preserve its return address if needed later.
- Commonly saves `x30` on the stack.

## Practical Rule

If a function uses `bl` and still needs its caller's return address, it must save `x30` before the nested call.

# Arguments and Return Values

# Integer Argument Registers

## AArch64 Procedure Call Standard

For ordinary integer and pointer arguments, the first eight arguments are passed in x0–x7.

| Register | Common role at a function call                                                |
|----------|-------------------------------------------------------------------------------|
| x0–x7    | Integer/pointer arguments. x0 also holds the main integer return value.       |
| x8       | Indirect result location in some ABI-defined cases.                           |
| x9–x15   | Temporary registers available for general use.                                |
| x16, x17 | Intra-procedure-call scratch registers used by veneers/linker-generated code. |

## Scope

This lecture focuses on integer and pointer arguments. Floating-point/SIMD argument registers are part of the ABI but are not the main topic here.

# Return Values

## Common Integer Return

`x0 = return value`

- A simple integer or pointer return value is placed in `x0`.
- Some larger or multi-register return values may use additional ABI rules.
- The caller reads the return value after the callee executes `ret`.
- If the caller needs an argument value after the call, it must preserve it if the register is caller-saved.

## Example

```
bl abs_value  
str x0, [x20]
```

# Example Function Call

C

```
z = add2(a, b);
```

- First argument: a
- Second argument: b
- Return value assigned to z

AArch64 Shape

```
mov x0, x19  
mov x1, x20  
bl add2  
mov x21, x0
```

The example assumes a, b, and z are being kept in x19, x20, and x21.

# Simple Leaf Function

## C Function

```
long add2(long a, long b)
return a + b;
```

## AArch64 Function Body

```
add2:
add x0, x0, x1
ret
```

- x0 and x1 arrive holding the two arguments.
- The sum is written to x0, the return-value register.
- No stack frame is required for this simple leaf function.

# Register Preservation Rules

# Caller-Saved and Callee-Saved Registers

| Category      | Registers | Meaning                                                                               |
|---------------|-----------|---------------------------------------------------------------------------------------|
| Caller-saved  | x0–x18    | A call may overwrite them. The caller saves any values it still needs.                |
| Callee-saved  | x19–x28   | The callee must restore these before returning if it changes them.                    |
| Frame pointer | x29       | Commonly used to reference the current stack frame.                                   |
| Link register | x30       | Holds the return address after <code>bl</code> ; non-leaf functions commonly save it. |

## Core Distinction

Caller-saved values may be destroyed by a call; callee-saved values must be restored by the function that changes them.

# Caller-Saved Example

## Situation

The caller has a live value in x0, but also needs to call another function. Since x0 is caller-saved, the caller must protect it if the value is still needed.

```
str x0, [sp, #-16]!  
bl helper  
ldr x0, [sp], #16
```

- This example reserves 16 bytes to preserve stack alignment.
- A compiler may use a different register or frame layout.
- The principle is that the caller saves caller-saved values it still needs.

# Callee-Saved Example

## Situation

A function wants to use x19. Because x19 is callee-saved, the function must restore the caller's original x19 before returning.

```
stp x19, x30, [sp, #-16]!  
// use x19 inside function  
ldp x19, x30, [sp], #16  
ret
```

## Preservation Rule

The caller should see the same x19 value after the function returns as before the call began.

# Common Register Roles

| Register(s) | Typical ABI role                                                         |
|-------------|--------------------------------------------------------------------------|
| x0–x7       | Integer/pointer arguments and return values.                             |
| x9–x15      | Temporary caller-saved registers.                                        |
| x19–x28     | Callee-saved registers.                                                  |
| x29         | Frame pointer, when a frame pointer is used.                             |
| x30         | Link register, holding a return address after bl.                        |
| sp          | Stack pointer; must satisfy the required alignment at public interfaces. |

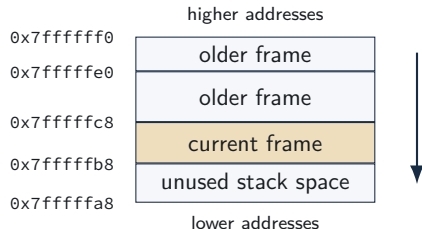
# Stack Frames

# The Stack Grows Down

- The stack is a memory region used for call-related temporary storage.
- On AArch64 systems using the standard ABI, `sp` is kept 16-byte aligned at function boundaries.
- Pushing data usually subtracts from `sp`.
- Popping data usually adds back to `sp`.

## Direction

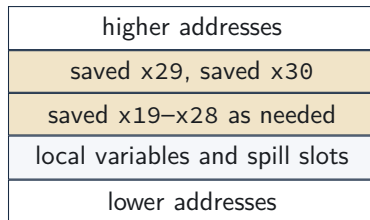
The stack grows toward lower addresses, so allocating stack space means decreasing `sp`.



illustrative only — frame sizes vary by function

# Stack Frame Contents

- Saved return address when x30 must be preserved.
- Saved frame pointer when x29 is used.
- Saved callee-saved registers used by the function.
- Local variables or spill slots that do not fit in registers.
- Outgoing stack arguments when register arguments are insufficient.



sp after allocation

# Function Prologue

## Purpose

A prologue prepares the callee's execution environment before the function body runs.

```
    stp x29, x30, [sp, #-16]!  
    mov x29, sp
```

- The first instruction allocates 16 bytes and stores old x29 and x30.
- `mov x29, sp` establishes a frame pointer for this frame.
- Additional stack allocation may follow if locals or spills are needed.

## Compiler Variation

Optimized code may omit x29 as a frame pointer or use a different frame layout, while preserving ABI obligations.

# Function Epilogue

## Purpose

An epilogue restores the caller's environment before returning.

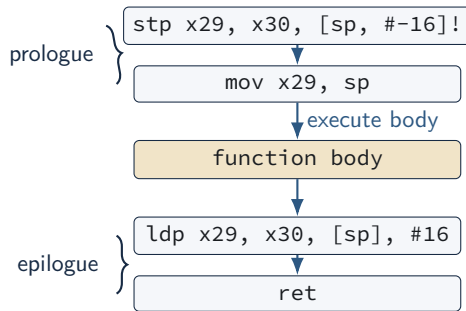
```
ldp x29, x30, [sp], #16  
ret
```

- The load pair restores the previous frame pointer and return address.
- Post-indexing adds 16 to sp, releasing the stack space.
- `ret` branches to the restored return address in x30.

## Stack Balance

A function that subtracts from sp must add the same amount back before returning.

# Prologue and Epilogue Diagram



## Function Entry and Exit

The prologue establishes the function's stack-frame state before the body executes. The epilogue reverses those changes, restores the saved return state, and then `ret` transfers control back to the caller.

# Local Stack Storage

## When Registers Are Not Enough

A function may allocate additional stack space for local arrays, temporary spill slots, or values whose address must be taken.

```
sub sp, sp, #32
str x0, [sp, #16]
ldr x1, [sp, #16]
add sp, sp, #32
```

- Stack space is usually allocated in multiples that preserve alignment.
- Positive offsets from `sp` or `x29` can address slots in the frame.
- The frame layout is a compiler convention, not an A64 instruction requirement.

# Function Calls and Recursion

# Leaf Function without a Stack Frame

## Leaf Function

A leaf function calls no other function. If it only uses caller-saved registers and needs no local stack storage, it may avoid stack use entirely.

```
double_it:  
    add x0, x0, x0  
    ret
```

- Argument arrives in x0.
- Return value is written back into x0.
- x30 is not overwritten because this function makes no nested calls.

# Non-Leaf Function Saves the Link Register

## Nested Call

A non-leaf function must preserve its own return address before executing another `bl` if it needs to return to its caller afterward.

```
outer:  
    stp x29, x30, [sp, #-16]!  
    mov x29, sp  
    bl inner  
    add x0, x0, #1  
    ldp x29, x30, [sp], #16  
    ret
```

## Reason

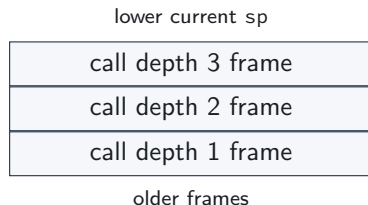
The `bl inner` instruction overwrites `x30`; therefore `outer` saves its original `x30` first.

# Recursion Requires Separate Frames

## Recursive Calls

Each recursive call needs its own preserved state: return address, saved registers, and local values that must survive the nested call.

- One invocation cannot reuse another invocation's saved return address.
- Local variables from different recursion depths must remain distinct.
- Stack frames naturally provide separate storage per invocation.



# Factorial Control Structure

## C

```
long fact(long n) {  
    if (n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

## Required State

- original  $n$ , needed after recursive call
- return address in  $x30$
- stack frame for each active invocation
- return value from recursive call in  $x0$

## Pedagogical Point

Recursion is not special hardware. It is ordinary calls plus separate stack frames for each active invocation.

# Recursive Function Skeleton

```
fact:
    stp x29, x30, [sp, #-32]!
    mov x29, sp
    str x0, [sp, #16]
    cmp x0, #1
    b.gt recurse
    mov x0, #1
    b done
recurse:
    sub x0, x0, #1
    bl fact
    ldr x1, [sp, #16]
    mul x0, x1, x0
done:
    ldp x29, x30, [sp], #32
    ret
```

# Common Procedure Bugs

- Forgetting that `bl` overwrites `x30`.
- Changing `x19–x28` without restoring them before returning.
- Returning with `sp` different from its value at function entry.
- Assuming caller-saved registers survive a call.
- Using `w` registers for pointer values that require 64-bit addresses.
- Saving one 64-bit register with only an 8-byte stack adjustment, breaking 16-byte stack alignment at an interface.

## Debugging Order

Check the return address, then stack balance, then register-preservation rules.

# Summary and Self-Test

## Topic 8 Summary

- `bl` calls a function and writes the return address into `x30`.
- `ret` returns to the address held in `x30` by default.
- `x0–x7` pass common integer/pointer arguments; `x0` holds the common integer/pointer return value.
- Caller-saved registers may be overwritten by a call; callee-saved registers must be restored if changed.
- Stack frames preserve return addresses, saved registers, and local storage.
- Leaf functions may avoid stack frames; non-leaf and recursive functions often need them.

# Self-Test 1/2

- ❶ What does `bl` function write into `x30`?
- ❷ Why can a non-leaf function not blindly trust `x30` after it calls another function?
- ❸ Which registers commonly carry the first eight integer or pointer arguments?
- ❹ Where is a simple integer return value normally placed?
- ❺ What is the difference between caller-saved and callee-saved registers?

## Self-Test 2/2

- 6 Why does `stp x29, x30, [sp, #-16]!` both allocate stack space and save registers?
- 7 What does `ldp x29, x30, [sp], #16` do before the final `ret`?
- 8 Why must `sp` be restored before returning to the caller?
- 9 Why can a small leaf function sometimes avoid a stack frame?
- 10 Why does recursion require a separate stack frame for each active call?

# Register Preservation Exercise

## Classify Each Register

For each register, state whether it is mainly used for argument/return, caller-saved temporary use, callee-saved preservation, frame pointer, link register, or stack pointer.

| Register | Classification |
|----------|----------------|
| x0       |                |
| x9       |                |
| x19      |                |
| x29      |                |
| x30      |                |
| sp       |                |

## Machine Instruction Encoding

Topic 9 moves from assembly-level procedure structure to the binary encoding of selected A64 instructions.

- Instruction fields and bit positions
- Register-number encoding
- Immediate fields and signed offsets
- Branch target encodings
- Relationship between assembly syntax and machine words